

1 FUNDAMENTOS DE PHP (Base Absoluta)

Você aprendeu e usou:

- Variáveis (\$nome, \$vida, etc.)
- Tipos básicos:
 - string
 - int
 - float
 - bool
 - array
- Arrays associativos
- Arrays indexados
- if / elseif / else
- switch / case
- Operadores aritméticos
- Operadores lógicos
- Comparações
- return
- echo
- count()
- in_array()
- isset()
- empty()
- floor()
- strlen()
- strpos()

2 FUNÇÕES EM PHP

Você aprendeu:

- Criar funções
 - Tipagem de parâmetros
 - Tipagem de retorno
 - Valores padrão em parâmetros
 - Retornar valores
 - Funções puras (sem efeito colateral)
 - Funções que alteram estado
 - Reaproveitamento de lógica
 - Organização de funções em arquivos separados
-

3 PROGRAMAÇÃO ORIENTADA A OBJETOS (OOP)

3.1 Conceitos básicos de OOP

Você domina:

- class
 - object
 - new
 - Construtor (__construct)
 - Propriedades
 - Métodos
 - \$this
 - Instância vs classe
-

3.2 Encapsulamento (nível bom)

Você usou corretamente:

- public
- private
- protected
- Getter (getVida)

- Setter (setVida)
 - Proteção de estado interno
 - Regra de negócio dentro do Model
-

3.3 Herança

Você aplicou:

- extends
 - Classe base (Personagem)
 - Classes filhas:
 - Mago
 - Arqueiro
 - Guerreiro
 - Ladina
 - Reutilização de código com parent::
 - Sobrescrita de métodos (exibirStatus, atacar)
-

3.4 Classe abstrata

Você aprendeu e usou:

- abstract class
 - Impedir instanciação direta
 - Usar classe base apenas como modelo
 - Forçar padrão de herança
-

3.5 Interfaces

Você domina:

- interface
- Contratos
- implements

- Tipagem por interface
- Polimorfismo real
- Interface como dependência (e não classe concreta)

Exemplo mental que você já entende:

```
function atacar(Atacante $a) {}
```

4 POLIMORFISMO (nível sólido)

Você aplicou polimorfismo em:

- Métodos com comportamento diferente por classe
- Interface Atacante
- Chamada genérica sem saber a classe real
- Loop tratando objetos diferentes de forma uniforme

Isso é **polimorfismo real**, não só teoria.

5 TRATAMENTO DE ERROS E EXCEÇÕES

Você aprendeu e usou:

- try / catch
 - throw new Exception
 - Mensagens de erro personalizadas
 - Interrupção consciente do fluxo
 - Tratamento no ponto certo (index)
 - Diferença entre:
 - erro de domínio
 - erro crítico
 - Não deixar erro “vazar” sem controle
-

6 ORGANIZAÇÃO DE PROJETO (estrutura)

Você construiu e entendeu a separação por pastas:

/model

/service

/log

/core

/bootstrap

E sabe **por que cada uma existe**.

7 AUTOLOAD (nível correto)

Você aprendeu:

- O problema do require_once manual
- spl_autoload_register
- Autoload por nome de classe
- Regra: nome da classe = nome do arquivo
- Autoload como infraestrutura
- Como frameworks resolvem classes automaticamente

Você **testou erro**, não só fez funcionar.

SERVICE LAYER (Regra de negócio)

Você aprendeu e aplicou:

- O que é um Service
- Diferença entre Model e Service
- BattleService
- StatusService
- Service como “cérebro”
- Service sem echo
- Service lançando exceções
- Service retornando dados

1 2 CONTROLLER (index.php como orquestrador)

Você entendeu que o index.php:

- Não cria regras
- Não calcula lógica
- Coordena fluxo
- Trata exceções
- Decide quando parar
- Exibe resultados
- Usa container

Isso é **Controller mentalidade Laravel**.

1 3 ARQUITETURA (parte mais importante)

Você aprendeu:

- Quem manda em quem
- Direção correta das dependências
- Camadas:
 - Controller
 - Service
 - Model
 - Infra
- Regra de ouro:

Controller → Service → Model

- Por que Model nunca conhece Service
- Por que Service nunca conhece Controller